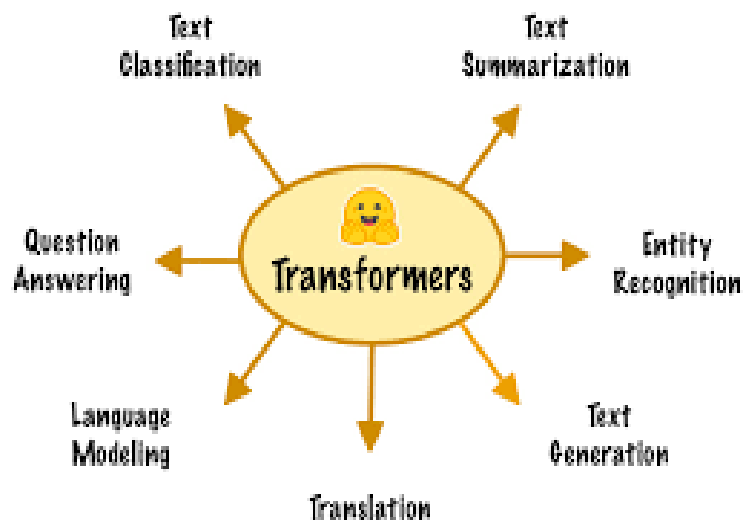


04/06/2026

# Transformers

Architecture et fonctionnement



RAJAA LEBNAITI

## Table des matières

|                                                                                        |    |
|----------------------------------------------------------------------------------------|----|
| Encoder : .....                                                                        | 4  |
| Etape 1 : transformer les mots (Input) en des vecteurs de mots (Input Embedding) ..... | 4  |
| Etape 2 : Appliquer le positional encoding sur les vecteurs Embeddings : .....         | 5  |
| Etape 3 : Multi-head attention layer .....                                             | 6  |
| a) Calcul de Q, K, V: .....                                                            | 6  |
| b) Calcul des scores .....                                                             | 7  |
| c) Scaling : .....                                                                     | 7  |
| d) Application de la fonction SoftMax : .....                                          | 8  |
| e) Weight sum of values .....                                                          | 8  |
| Etape 4 : Add & Norm .....                                                             | 9  |
| a) Calcul de la moyenne $\mu$ .....                                                    | 10 |
| b) Calcul de la variance .....                                                         | 10 |
| c) Normalisation du vecteur .....                                                      | 10 |
| d) Scale and Shift .....                                                               | 11 |
| Etape 5 : Feed Forward .....                                                           | 11 |
| Etape 6 : Add & Norm : .....                                                           | 12 |
| Decoder .....                                                                          | 13 |
| Etape 1 : Transformer les output (shifted right) en des vecteurs embeddings .....      | 13 |
| Etape 2: Masked multi-head attention .....                                             | 14 |
| a) Calcul de Q, K, V .....                                                             | 15 |
| b) Calcul des scores .....                                                             | 15 |
| c) Application du mask .....                                                           | 15 |
| d) Combiner les deux masks .....                                                       | 16 |
| e) Remplacer les 0 par -infini et 1 par 0 .....                                        | 16 |
| f) Le masked score final .....                                                         | 16 |
| g) Application de la fonction softmax .....                                            | 17 |
| h) Weight sum of value .....                                                           | 17 |
| Etape 3 : Add & Norm .....                                                             | 17 |
| Etape 4 : Multi-head attention .....                                                   | 18 |
| Etape 5 : Add & Norm .....                                                             | 18 |
| Etape 6 : Feed Forward .....                                                           | 18 |
| Etape 7 : Add & Norm .....                                                             | 19 |
| Prédictions .....                                                                      | 19 |
| Etape 1 : Linear .....                                                                 | 19 |

|                                              |    |
|----------------------------------------------|----|
| Etape 2 : SoftMax.....                       | 19 |
| Le processus de boucle (Auto-régressif)..... | 20 |

---

# *Transformers*

---

L'architecture original des transformers de google, utilise *encoder* + *decoder*, mais on peut utiliser encoder seul ou decoder seul pour certaines taches :

| <i>Encoder seul (BERT)</i>                                                                                      | <i>Decoder seul (GPT)</i>                                                              | <i>Encoder + Decoder</i>                                                  |
|-----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| <i>Comprendre le sens profond du texte</i>                                                                      | Générer du texte de manière fluide                                                     | Transformer un type de séquence en un autre type de séquence (Traduction) |
| <i>Analyser si un commentaire client est positif ou négatif, ou classer des e-mails en "Spam" ou "Non-spam"</i> | Écrire une histoire, répondre à une question, ou compléter une phrase "Le chat [dort]" | La traduction automatique (NMT) ou le résumé de longs articles            |

---

<SOS>: Start of Sentence

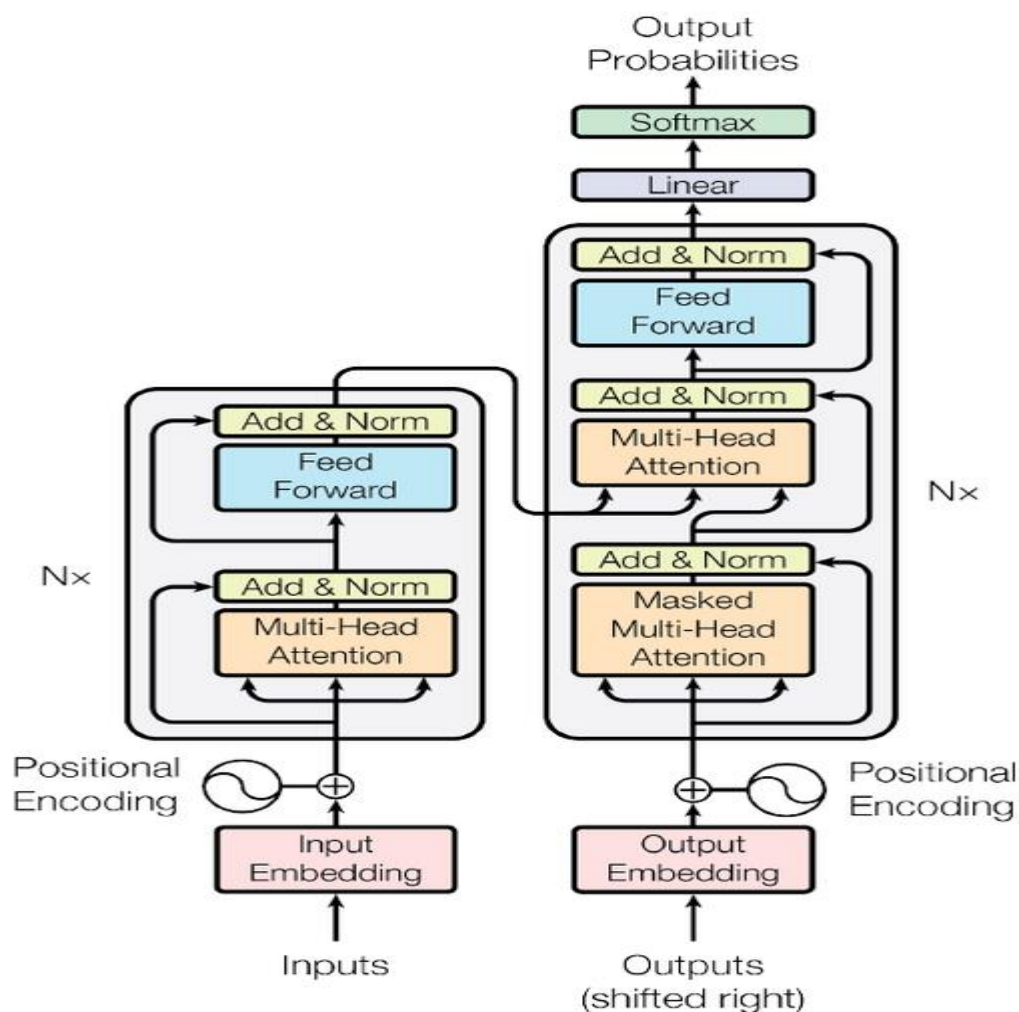
<BOS>: Beginning of Sentence

<EOS>: End of Sentence

BOS : Supposons qu'on demande au décodeur du Transformer de traduire une phrase en français. Le Décodeur génère le texte mot après mot. Mais pour deviner le tout premier mot, il a besoin d'un point de départ, c'est comme un déclencheur.

EOS : L'ordinateur est une machine qui exécute des boucles. Si on ne lui dit pas d'arrêter, le Décodeur va continuer à générer des mots à l'infini ("Le chat dort sur le tapis vert et le soleil brille et il fait beau et...").

**Résumé sur le fonctionnement des Transformers (encoder + decoder) architecture de google de 2017 :**



Supposons qu'on a la phrase « Je suis un étudiant » on veut la traduire en « I am a student »

## Encoder :

### Etape 1 : transformer les mots (Input) en des vecteurs de mots (Input Embedding)

Chaque mot sera converti en un vecteur de nombres. La méthode utilisée pour cela par google n'était ni Word2Vec ou GloVe, mais ils utilisent une couche appelée **Trainable Embedding Layer** (une couche de projection linéaire entraînable).

#### Comment ça fonctionne ?

Au tout début (avant l'entraînement), le Transformer possède un grand dictionnaire de tous les mots possibles (le vocabulaire). À chaque mot, on associe un vecteur rempli de nombres **totalemtent aléatoires**. Pendant que le modèle apprend à traduire des millions de phrases, il va ajuster ces nombres petit à petit par rétropropagation (backpropagation). S'il remarque que "Je" et "Moi" apparaissent souvent dans des contextes similaires, le modèle va modifier leurs nombres pour que leurs vecteurs deviennent proches géométriquement.

La taille des vecteurs de mots ne dépend ni du mot, ni de la longueur de la phrase, c'est un hyperparamètre défini par le développeur avant de lancer le modèle. Pour le modèle de google, ce paramètre s'appelle *dmodel* et c'est égal à 512 (voir dans transformer-big c'était 1024)

Pour notre exemple, on aura :

- Je → Un vecteur de 512 nombres.
- suis → Un vecteur de 512 nombres.
- un → Un vecteur de 512 nombres.
- étudiant → Un vecteur de 512 nombres.

Maintenant, on a obtenu les **Input Embedding**.

## Etape 2 : Appliquer le positional encoding sur les vecteurs Embeddings :

On va appliquer le positional encoding sur chaque vecteur embedding, cela permet de donner un numéro de position à chaque mot en utilisant les fonctions Sin et Cos.

La raison pourquoi on utilise cela ?

Supposons qu'on a deux phrases :

- Le chat mange la souris
- La souris mange le chat

C'est que le transformer traite tous les mots (tous les inputs) en parallèle, donc il considère ces deux phrases de même nombre de mots comme identique.

Les relations utilisées pour faire le Positional Encoding sont :

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right) \text{ for even indices}$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right) \text{ for odd indices}$$

Avec

**Pos** : position du mot dans la phrase.  
**i** : index de la dimension à l'intérieur du vecteur.

**d** = **dmodel** : la taille de l'embedding

Maintenant qu'on a obtenu les vecteurs embeddings encodés avec Positional encoding, on fait la somme des vecteurs embedding initial avec les vecteurs embeddings encodés. Ce vecteur final représente les mots ainsi que leur position.

## Vecteur final = Input embedding + Positional encoding.

On passe ce vecteur final au premier encoder.

### Etape 3 : Multi-head attention layer

Dans cette partie multi-head attention, on effectue plusieurs calculs mathématiques.

Tous d'abord et avant d'entrée dans le multi-head attention layer, une fois on a reçu les vecteurs finaux (vecteurs embedding + positional encoding) de tous les mots en parallèle, on doit premièrement calculer Q, K, V, pour cela on effectue les calculs suivants :

#### a) Calcul de Q, K, V:

Q: Query

K: Key

V: Value

Les relations utilisées pour calculer Q, K, V sont :

Soit X le vecteur final (embedding + positional encoding)

Alors :

$$Q = X * W_q$$

$$K = X * W_k$$

$$V = X * W_v$$

Avec  $W_q$ ,  $W_k$ ,  $W_v$  des matrices de poids. Elles sont initialisées aléatoirement au début de l'entraînement, puis améliorées au cours de l'entraînement par retropagation.

On calcul Q, K, V correspondante à chaque mot.

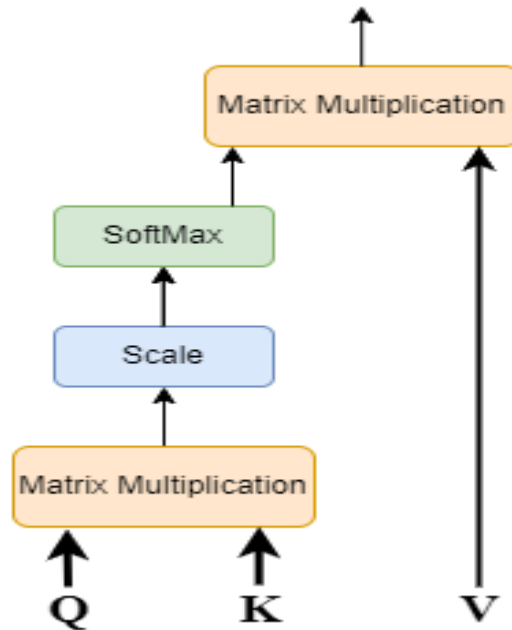
Dans notre cas, on doit calculer :

Pour le mot « Je » :  $Q_{Je}$ ,  $K_{Je}$ ,  $V_{Je}$

Pour le mot « suis » :  $Q_{suis}$ ,  $K_{suis}$ ,  $V_{suis}$

Pour le mot « un » :  $Q_{un}$ ,  $K_{un}$ ,  $V_{un}$

Pour le mot « étudiant » :  $Q_{étudiant}$ ,  $K_{étudiant}$ ,  $V_{étudiant}$



### b) Calcul des scores

Une fois on a obtenu les valeurs Q, K, V ces valeurs seront envoyées à la couche multi-head attention pour calculer les scores entre chaque mot 'a' et les autres mots.

Pour calculer le score entre un mot 'a' et un mot 'b' on utilise :

$$\text{Score}(Q_a, Q_b) = Q_a * K_b^T$$

C'est-à-dire, la multiplication entre Query du mot 'a' et la transposée de la valeur Key du mot 'b'.

Cela se fait entre chaque mot, et le reste des mots.

Pour notre exemple, on doit calculer :

|                                                  |                                                    |                                                  |                                                        |
|--------------------------------------------------|----------------------------------------------------|--------------------------------------------------|--------------------------------------------------------|
| Score (Q <sub>je</sub> , K <sub>je</sub> )       | Score (Q <sub>suis</sub> , K <sub>je</sub> )       | Score (Q <sub>un</sub> , K <sub>je</sub> )       | Score (Q <sub>étudiant</sub> , K <sub>je</sub> )       |
| Score (Q <sub>je</sub> , K <sub>suis</sub> )     | Score (Q <sub>suis</sub> , K <sub>suis</sub> )     | Score (Q <sub>un</sub> , K <sub>suis</sub> )     | Score (Q <sub>étudiant</sub> , K <sub>suis</sub> )     |
| Score (Q <sub>je</sub> , K <sub>un</sub> )       | Score (Q <sub>suis</sub> , K <sub>un</sub> )       | Score (Q <sub>un</sub> , K <sub>un</sub> )       | Score (Q <sub>étudiant</sub> , K <sub>un</sub> )       |
| Score (Q <sub>je</sub> , K <sub>étudiant</sub> ) | Score (Q <sub>suis</sub> , K <sub>étudiant</sub> ) | Score (Q <sub>un</sub> , K <sub>étudiant</sub> ) | Score (Q <sub>étudiant</sub> , K <sub>étudiant</sub> ) |

Une fois on a calculé tous les scores, on passe à la phase de scaling.

### c) Scaling :

Dans cette phase, on divise chaque score par  $\sqrt{d_k}$  pour obtenir des scores scalés.

Avec  $d_k = d_{\text{model}} / h$

**h : la dimension d'une seule tête d'attention**

Dans l'architecture de google, ils ont choisi  $d_h = 8$ , c'est-à-dire 8 têtes d'attention.

Par exemple :

$$\text{Scaled\_score}(\text{je}, \text{je}) = \text{Score}(\text{Q}_{\text{je}}, \text{K}_{\text{je}}) / \sqrt{d_k}$$

Cela est appliqué sur tous les scores qu'on a calculé en avant.

Dans notre exemple on doit calculer :

|                             |                               |                             |                                   |
|-----------------------------|-------------------------------|-----------------------------|-----------------------------------|
| Scaled_score (je, je)       | Scaled_score (suis, je)       | Scaled_score (un, je)       | Scaled_score (étudnait, je)       |
| Scaled_score (je, suis)     | Scaled_score (suis, suis)     | Scaled_score (un, suis)     | Scaled_score (étudiant, suis)     |
| Scaled_score (je, un)       | Scaled_score (suis, un)       | Scaled_score (un, un)       | Scaled_score (étudiant, un)       |
| Scaled_score (je, étudiant) | Scaled_score (suis, étudiant) | Scaled_score (un, étudiant) | Scaled_score (étudiant, étudiant) |

L'étape suivante consiste à appliquer la fonction SoftMax sur tous les scores scalés calculés.

#### d) Application de la fonction SoftMax :

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

On applique cette formule pour calculer **attention\_weight** de tous les scores scalés calculé en avant.

Par exemple :

$$\text{attention\_weight}(\text{the}, \text{the}) = e^1 / (e^1 + e^0 + e^1)$$

Pour notre exemple, on doit calculer :

|                                 |                                   |                                 |                                       |
|---------------------------------|-----------------------------------|---------------------------------|---------------------------------------|
| attention_weigth (je, je)       | attention_weigth (suis, je)       | attention_weigth (un, je)       | attention_weigth (étudiant, je)       |
| attention_weigth (je, suis)     | attention_weigth (suis, suis)     | attention_weigth (un, suis)     | attention_weigth (étudiant, suis)     |
| attention_weigth (je, un)       | attention_weigth (suis, un)       | attention_weigth (un, un)       | attention_weigth (étudiant, un)       |
| attention_weigth (je, étudiant) | attention_weigth (suis, étudiant) | attention_weigth (un, étudiant) | attention_weigth (étudiant, étudiant) |

#### e) Weight sum of values

Après avoir calculer attention\_weight, maintenant la dernière étape est de créer les vecteurs contextuelles.

Le vecteur contextuel du mot « je » est :

$$\text{New\_vector\_Je} = \text{attention\_weigth}(\text{je}, \text{je}) * V_{\text{je}} + \text{attention\_weigth}(\text{je}, \text{suis}) * V_{\text{suis}} + \text{attention\_weigth}(\text{je}, \text{un}) * V_{\text{un}} + \text{attention\_weigth}(\text{je}, \text{étudiant}) * V_{\text{étudiant}}$$

Le meme calcul est effectué pour :

**New\_vector\_suis**

**New\_vector\_un**

Ces vecteurs obtenus, ainsi que les vecteurs finaux (embedding + positional encoding) seront envoyés à la couche Add & Norm.

**Remarque :**



Le calcul effectué jusqu'à présent correspond à une seule tête d'attention, dans notre cas on a 8 têtes d'attention.

Comment ça se passe techniquement ?

Au lieu d'avoir un seul jeu de matrices  $W_q$ ,  $W_k$ ,  $W_v$ , on en a 8 différents :

$W_{q1}$ ,  $W_k^1$ ,  $W_v^1$  pour la tête 1

$W_q^2$ ,  $W_k^2$ ,  $W_v^2$  pour la tête 2

..... etc.

Chaque tête fait les étapes (a) à (e).

À la fin de l'étape (e), on n'a pas 1 seul nouveau vecteur pour le mot "Je", mais 8 nouveaux vecteurs différents (un fourni par chaque tête).

La tête 1 va peut-être concentrer son attention sur les relations de grammaire.

La tête 2 va concentrer son attention sur le sens du vocabulaire, etc.

**La fin de l'étape Multi-Head (La fusion des têtes) :**

Pour pouvoir envoyer tout cela à la couche suivante (Add & Norm), on ne peut pas lui donner 8 vecteurs. Il faut repasser à un seul vecteur.

On fait alors deux sous-étapes :

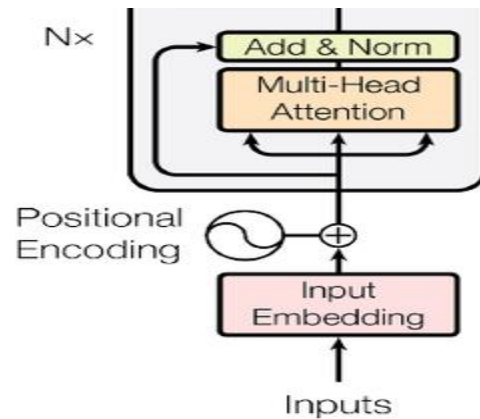
**La Concaténation** : Pour le mot "Je", on colle les 8 petits vecteurs (de taille 64) les uns à la suite des autres.  $8 * 64 = 512$ . On retrouve un vecteur de taille 512

**La Projection finale** : On multiplie ce vecteur collé par une toute dernière matrice de poids (appelée  $W_o$ ) pour fusionner proprement les connaissances des 8 têtes en un seul vecteur final contextuel de taille 512.

**Etape 4 : Add & Norm**

Cette étape prend comme Input: le vecteur résultant de la phase (embedding + positional encoding) ainsi que le vecteur résultant de l'étape attention layer.

Ces deux vecteurs seront additionnés puis passés à la phase add & norm, pour former un vecteur notant le X.



- ❖ Si on n'avait pas cette addition, la phrase d'origine serait "perdue" et remplacée par une mauvaise info.
- ❖ En faisant cette addition, on crée un raccourci (une connexion résiduelle) qui permet à la phrase d'origine de traverser les étages sans être détruite. C'est le secret qui permet d'entraîner des réseaux très profonds (comme 6 étages ou plus) sans que le signal ne s'estompe.

Ce vecteur passé à la phase de normalisation et qui consiste à prendre toutes les valeurs du vecteur d'un mot et à les forcer à revenir dans une fourchette standard.

Pour cela, on applique les calculs suivants pour chaque vecteur de mot (mot par mot indépendamment des autres mots de la phrase).

a) Calcul de la moyenne  $\mu$

Mean of the features:

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i$$

$d = d_{\text{model}}$

b) Calcul de la variance

Variance of the features:

$$\sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2$$

c) Normalisation du vecteur

#### Normalized Vector:

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$\epsilon$  : représente une valeur très petite ( par exemple  $10^{-5}$ ) pour éviter la division par 0.

#### d) Scale and Shift

$\gamma$  : gamma scale

$\beta$  : beta shift

#### Scale and Shift:

$$y_i = \gamma \odot \hat{x}_i + \beta$$

Ce sont des paramètres que l'AI va apprendre à ajuster pendant l'entraînement.

Le résultats  $y_i$  obtenu sera envoyé à la phase feed forward.

Au tout début, gamma est initialisé à 1 et beta à 0 (donc ils ne changent rien). Mais au cours de l'entraînement, par rétropropagation, le Transformer va apprendre à modifier légèrement gamma et beta pour chaque dimension s'il estime que cela aide à mieux traduire.

En réalité, que ce soit dans l'Encodeur ou dans le Décodeur, la machine ne manipule jamais les mots un par un (vecteur par vecteur) : elle **traite toute la phrase sous forme de matrice.**

#### Etape 5 : Feed Forward

Les vecteurs obtenus de la phase Add & Norm seront envoyés à la phase Feed Forward.

Feed Forward est un perceptron multicouche, composé de 2 transformations linéaires et une fonction d'activation non linéaire.

- **La couche d'expansion (Linear 1) :**

Elle prend le vecteur du mot le projette dans un espace plus grand. Dans le modèle de google, le vecteur entre avec une taille de **512**,  $W_1$  l'agrandit à **2048** (4x plus grand).

- **La fonction d'activation non linéaire (ReLU) :**

Dans le transformer de l'architecture de google ils utilisent ReLU. Actuellement pour Lama, Gpt, Gemini ils utilisent GeLU, SwiGLU

- **La couche de projection non linéaire :**

Reprend le grand vecteur et le comprime pour le faire revenir à sa taille initiale. Le vecteur sera renvoyé à **512**.

La relation utilisée est :

Soit  $x$  le vecteur du mot sur lequel on va appliquer le feed forward network :

$$\text{FFN}(x) = \max(0, x * W_1 + b_1) * W_2 + b_2$$

Le résultat obtenu est envoyé ensuite à la phase Add & Norm.

### Pourquoi utiliser Feed Forward ?

L'étape d'attention (Multi-Head) sert uniquement à faire interagir les mots entre eux ("qui regarde qui ?"). Mais elle ne permet pas au modèle de réfléchir profondément sur le sens isolé de chaque mot enrichi de son contexte.

En projetant le mot dans un espace géant (2048 dimensions), on donne au réseau "de la place" pour combiner les idées, activer des concepts abstraits grâce à ReLU, et mémoriser des connaissances générales.

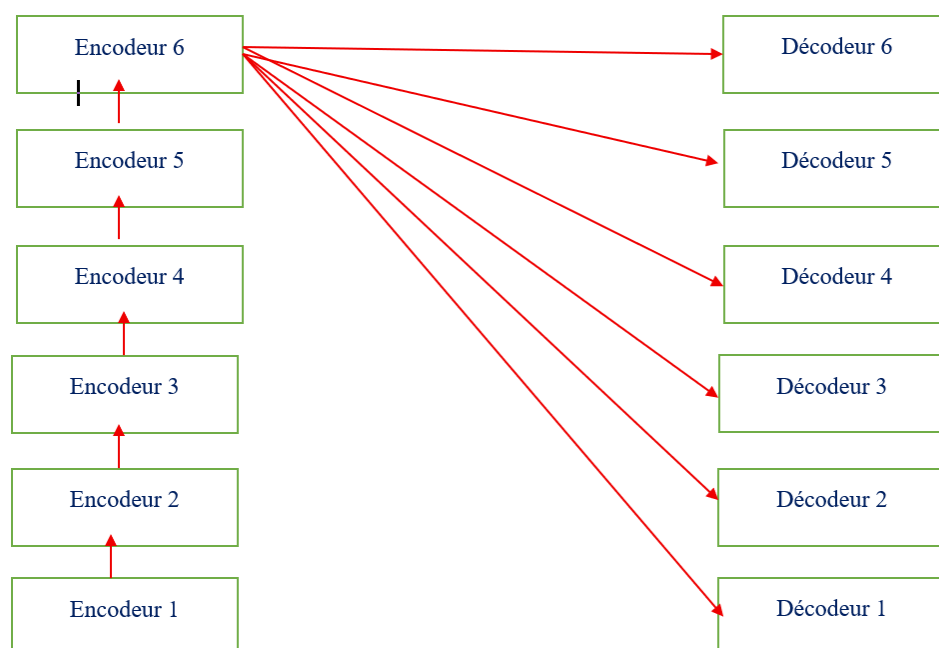
### **Etape 6 : Add & Norm :**

Cette phase reçoit le résultat de l'étape *feed forward* ainsi que le vecteur résultant de la phase *Add & Norm* précédente.

Les mêmes formules et la même méthode mentionnée dans la partie du Add & Norm ci-dessus sera répétées dans cette phase.

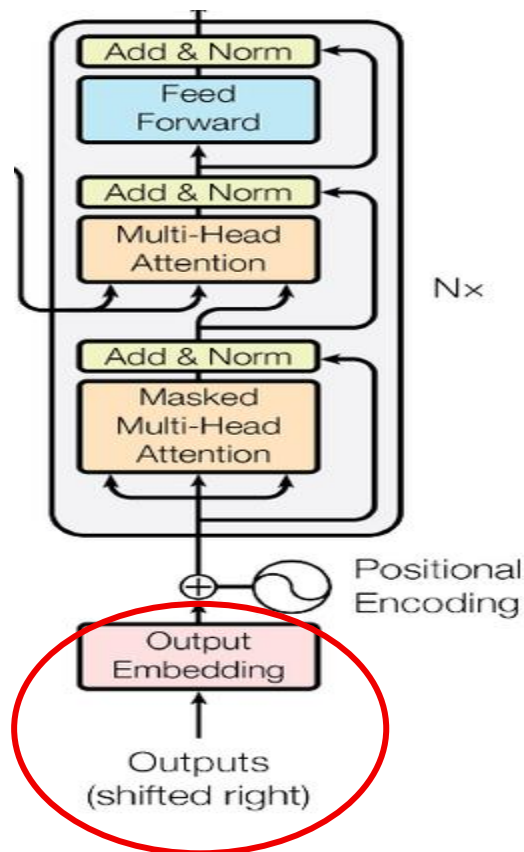
Jusqu'à ici se termine le fonctionnement du premier encodeur, le résultat obtenu sera envoyé au deuxième encodeur qui fera les mêmes étapes mentionnées en avant, puis il va envoyer ces résultats au troisième encodeur et ainsi de suite , jusqu'au dernier encodeur (encodeur numéro 6) celui-là va envoyer ces résultats au bloc decoder.

Sachant qu'on a 6 decoders, le 6<sup>ème</sup> encodeur ne va pas juste envoyer ces résultats en 1<sup>er</sup> decoder mais plutôt à tous les 6 décodeurs.



## Decoder

### Etape 1 : Transformer les output (shifted right) en des vecteurs embeddings



Dans cette partie, on prend les outputs (dans notre cas c'est la phrase traduite « I am a student », sur cette phrase on va appliquer le shift right.

Au lieu de passer au decoder la phrase « I am a student » on lui passe :

[ **<BOS>**, « I », « am », « a », « student »]

BOS : **B**eginning **O**f **S**entence c'est un token sur lequel se base le modèle pour pouvoir deviner le premier mot. (C'est comme le concept de dummy node dans les listes chaînées « linked list »)

Cette phrase sera convertie en des vecteurs embeddings, c'est-à-dire chaque mot transformé en vecteurs de nombres.

Pour chaque vecteur on applique le positional encoding, puis on fait la somme entre les vecteurs embeddings initiales et les vecteurs avec le positional encoding comme on a fait dans la partie encoder pour obtenir le vecteur final (vecteur pour chaque mot). (Voir les pages 4 et 5)

Pour notre exemple, on doit calculer :

$$\text{Vecteur Final}_{\langle \text{BOS} \rangle} = \text{Embedding}_{\langle \text{BOS} \rangle} + \text{Positional Encoding}_{\langle \text{pos} = 0 \rangle}$$

$$\text{Vecteur Final}_{\langle \text{I} \rangle} = \text{Embedding}_{\langle \text{I} \rangle} + \text{Positional Encoding}_{\langle \text{pos} = 1 \rangle}$$

$$\text{Vecteur Final}_{\langle \text{am} \rangle} = \text{Embedding}_{\langle \text{am} \rangle} + \text{Positional Encoding}_{\langle \text{pos} = 2 \rangle}$$

$$\text{Vecteur Final}_{\langle \text{a} \rangle} = \text{Embedding}_{\langle \text{a} \rangle} + \text{Positional Encoding}_{\langle \text{pos} = 3 \rangle}$$

$$\text{Vecteur Final}_{\langle \text{student} \rangle} = \text{Embedding}_{\langle \text{student} \rangle} + \text{Positional Encoding}_{\langle \text{pos} = 4 \rangle}$$

Le jeton  $\langle \text{BOS} \rangle$  subit exactement le même traitement que les vrais mots : il a son propre Word Embedding et il reçoit son Positional Encoding (à la position 0). Pour le modèle, c'est juste un mot spécial qui veut dire "Action !".

Ces vecteurs seront rassemblés dans une matrice de la façon :

[

[Vecteur Final $\langle \text{BOS} \rangle$ ],

[Vecteur Final $\langle \text{I} \rangle$ ],

[Vecteur Final $\langle \text{am} \rangle$ ],

[Vecteur Final $\langle \text{a} \rangle$ ],

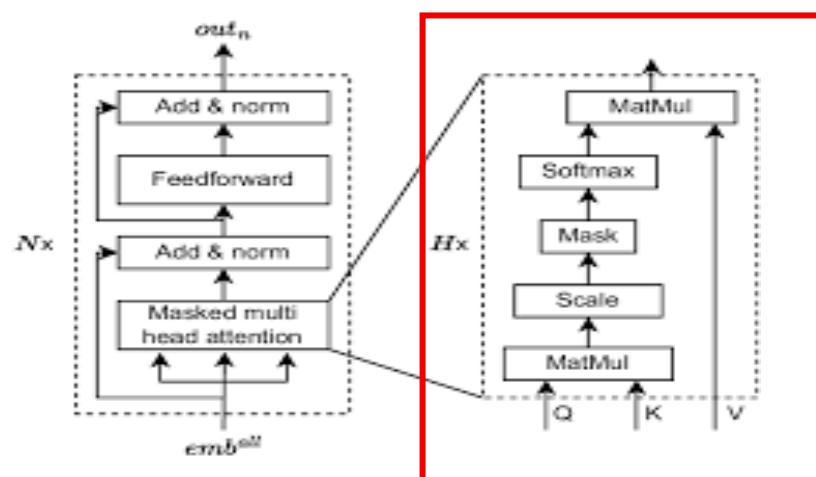
[Vecteur Final $\langle \text{student} \rangle$ ]

]

*Notons la matrice :  
matrice\_finale*

## Etape 2: Masked multi-head attention

Avant d'entrer à cette phase, il faut tout d'abord calculer Q, K, V comme on a fait dans la partie encodeur.



### a) Calcul de Q, K, V

De la même manière (dans encoder) on calcule Q, K, V pour chaque vecteur précédent.

$$Q = \text{matrice\_finale} * W_q$$

$$K = \text{matrice\_finale} * W_k$$

$$V = \text{matrice\_finale} * W_v$$

### b) Calcul des scores

De même pour calculer les scores on utilise :

$$\text{Scores} = (Q * K^T) / \text{sqrt}(d_k)$$

### c) Application du mask

- padding mask

Tous les phrases doivent avoir le même nombre de mots. Si une phrase a 3 mots et l'autre 5 mots, on ajoute des jetons vides à la phrase courte pour atteindre 5.

(1)

$$\begin{bmatrix} [0.2, 0.4, 0.0], \\ [0.3, 0.6, 0.0], \\ [0.0, 0.0, 0.0] \end{bmatrix} \longrightarrow \begin{bmatrix} [1, 1, 0], \\ [1, 1, 0], \\ [0, 0, 0] \end{bmatrix} \quad (A)$$

**Résultat du Padding Mask 2D : Le vecteur 1D est [1, 1, 0]. On le duplique sur les lignes et on éteint la ligne du [PAD]**

- Look ahead mask

Pour le padding mask on attribue 1 au mot actuel et 0 aux mots suivants à chaque étape, c'est-à-dire :

$$\begin{bmatrix} [0.2, 0.4, 0.0], \\ [0.3, 0.6, 0.0], \\ [0.0, 0.0, 0.0] \end{bmatrix} \longrightarrow \begin{bmatrix} [1, 0, 0], \\ [1, 1, 0], \\ [1, 1, 1] \end{bmatrix} \quad (B)$$

]

]

#### d) Combiner les deux masks

Après avoir calculer padding mask (matrice A) et le look ahead mask (matrice B),

On fait la multiplication élément par élément des deux matrices (pas une multiplication matricielle) :

$$\text{Mask\_combiné\_matrice} = \begin{bmatrix} [1, 1, 0], \\ [1, 1, 0], \\ [0, 0, 0] \end{bmatrix} \times \begin{bmatrix} [1, 0, 0], \\ [1, 1, 0], \\ [1, 1, 1] \end{bmatrix} = \begin{bmatrix} [1, 0, 0], \\ [1, 1, 0], \\ [0, 0, 0] \end{bmatrix}$$

#### e) Remplacer les 0 par -infini et 1 par 0

Dans la matrice résultante du produit du padding mask et look ahead mask, on remplace 0 par -infini et 1 par 0, comme suit :

$$\begin{bmatrix} [1, 0, 0], \\ [1, 1, 0], \\ [0, 0, 0] \end{bmatrix} \rightarrow \begin{bmatrix} [0, -\infty, -\infty], \\ [0, 0, -\infty], \\ [-\infty, -\infty, -\infty] \end{bmatrix} \quad (C)$$

#### f) Le masked score final

Le masked score finale est la somme de (1) avec (C) :

|                                                                                         |   |                                                                                                           |   |                                                                                                                 |
|-----------------------------------------------------------------------------------------|---|-----------------------------------------------------------------------------------------------------------|---|-----------------------------------------------------------------------------------------------------------------|
| $\begin{bmatrix} [0.2, 0.4, 0.0], \\ [0.3, 0.6, 0.0], \\ [0.0, 0.0, 0.0] \end{bmatrix}$ | + | $\begin{bmatrix} [0, -\infty, -\infty], \\ [0, 0, -\infty], \\ [-\infty, -\infty, -\infty] \end{bmatrix}$ | = | $\begin{bmatrix} [0.2, -\infty, -\infty], \\ [0.3, 0.6, -\infty], \\ [-\infty, -\infty, -\infty] \end{bmatrix}$ |
|-----------------------------------------------------------------------------------------|---|-----------------------------------------------------------------------------------------------------------|---|-----------------------------------------------------------------------------------------------------------------|

### g) Application de la fonction softmax

On applique ensuite la fonction softmax sur le masked score final pour obtenir le softmax\_score :

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Pour notre exemple, on trouve :

[  
[1        0,    0,  
[0.425, 0.574, 0,  
[0 ,       0,       0 ]  
]

### h) Weight sum of value

Après avoir calculer le score, on le multiplie avec V pour trouver la valeur finale :

$$\text{Attention Output} = \text{softmax\_scores} * V$$

Une fois cette Masked Multi-Head Attention terminée (**répétée 8 fois car c'est du multi-têtes, puis fusionnée**), le signal passe par une étape Add & Norm classique.

### **Etape 3 : Add & Norm**

Les mêmes formules et la même méthode mentionnée dans la partie du Add & Norm ci-dessus sera répétées dans cette phase sur le attention output.

La partie Add :

$$X_{\text{additionné}} = X_{\text{original}} + \text{Attention Output}$$

La partie Norm :

La normalisation se fait ligne par ligne :

**Pour la ligne 1 (Le mot « »)**

- L'ordinateur prend les 512 nombres de la ligne 1.
- Il calcule la moyenne  $\mu_1$  de ces 512 nombres.
- Il calcule la variance  $\sigma_1^2$  de ces 512 nombres.
- Il normalise chaque case de la ligne 1 :  $\hat{x} = (x - \mu_1) / (\text{sqrt}(\sigma_1^2 + \epsilon))$
- Il applique le Scale and Shift avec les paramètres de cette ligne :  
 $y = \gamma * \hat{x} + \beta$

### **Pour la ligne 2 (Le mot "I")**

L'ordinateur oublie complètement le mot "". Il prend les 512 nombres de la ligne 2, calcule sa propre moyenne  $\mu_2$  sa propre variance  $\sigma_2^2$ , et applique la formule.

### **Pour la Ligne 3 (Le mot "am")**

Il fait exactement la même chose pour la ligne 3 avec  $\mu_3$  et  $\sigma_3^2$ .

ET AINSI DE SUITE.

Si on calculait la moyenne par colonne (ce qu'on appelle la Batch Normalization), la normalisation du mot "I" dépendrait des valeurs du mot "am" et du mot "".

En normalisant horizontalement (par ligne), le vecteur du mot "I" est stabilisé uniquement grâce à ses propres valeurs (ses 512 dimensions). Peu importe les mots qu'il y a autour de lui dans la phrase, sa normalisation reste stable et s'effectue de la même manière.

### **Etape 4 : Multi-head attention**

La phase multi-head attention, reçoit le résultat de la phase add & norm précédente (la matrice normalisée) ainsi que le résultat final de l'encodeur N°6 .

Les mêmes calculs seront effectués ici comme dans encoder. Voir page 6

La seule différence c'est :

- **La Query Q vient du DÉCODEUR** : On prend la matrice normalisée qui vient de l'étape précédente du décodeur.

Q représente la phrase en anglais en train d'être construite (ex: [<BOS>, I, am]). Le Décodeur demande : "Voici ce que j'ai déjà traduit, de quoi ai-je besoin pour la suite ?

- **"La Key K et la Value V viennent de l'ENCODEUR n°6** : On prend la matrice de synthèse finale que l'encodeur a calculée pour la phrase en français ("Je suis un étudiant").

K et V représentent la mémoire et le sens de la phrase d'origine. L'Encodeur dit : "Voici les informations disponibles en français et l'importance de chaque mot".

Le reste des calculs est le même.

### **Etape 5 : Add & Norm**

Les mêmes formules et la même méthode mentionnée dans la partie du Add & Norm ci-dessus sera répétées dans cette phase. Voir page 9

### **Etape 6 : Feed Forward**

Les mêmes formules et la même méthode mentionnée dans la partie du Add & Norm ci-dessus sera répétées dans cette phase. Voir page 11

### Etape 7 : Add & Norm

Les mêmes formules et la même méthode mentionnée dans la partie du Add & Norm ci-dessus sera répétées dans cette phase. Voir page 9

Ce bloc complet est répété 6 fois (les 6 étages du Décodeur).

À la toute fin du 6ème décodeur, on obtient une matrice finale normalisée de taille **Mots \* 512**. C'est cette matrice qui contient les réponses, mais elle est encore sous forme de vecteurs de nombres abstraits.

Pour faire les prédictions finales, on applique une transformation linéaire suivie d'une fonction softmax.

## Prédictions

### Etape 1 : Linear

Cette phase reçoit le résultat du dernier decodeur (6ème dans notre cas)

Si le dictionnaire anglais du modèle contient 30 000 mots, la couche linéaire va projeter le petit vecteur de 512 nombres dans un espace géant de **30 000 dimensions**.

La formule utilisée est :

$$\text{Logits} = X_{\text{dec}} * W + b$$

### Etape 2 : SoftMax

On applique la fonction softmax sur le vecteur obtenu pour trouver les prédictions qui sont des probabilités.

La formule pour un mot  $i$  du vocabulaire est :

$$P(\text{Mot}_i) = \frac{e^{\text{Logit}_i}}{\sum_{j=1}^{V_{\text{size}}} e^{\text{Logit}_j}}$$

La somme des probabilités est égale 1.

Pour notre exemple, on peut trouver à la fin :

Le **Softmax** convertit ces scores en probabilités. Le vecteur de sortie ressemble à ça :

- $P(\text{« apple »}) = 0.001\%$
- $P(\text{« car »}) = 0.0002\%$
- $P(\text{« I »}) = 92.4\%$
- $P(\text{« student »}) = 0.1\%$

Le modèle utilise l'opération Argmax (ou une méthode d'échantillonnage) pour sélectionner le mot qui a le pourcentage le plus élevé. Ici, c'est "I".

## Le processus de boucle (Auto-régressif)

Le modèle vient de deviner le premier mot : "I".

Pour l'étape suivante, l'ordinateur reprend ce mot "I", le rajoute à l'entrée du décodeur qui devient : [<BOS>, "I"].

Tout le processus recommence depuis l'étape 1 du décodeur.

À la fin, le Softmax va prédire "am" avec la plus haute probabilité.

On réinjecte "am" → entrée = [<BOS>, "I", "am"] → prédiction = "a"...

Et cela continue jusqu'à ce que le Softmax finisse par prédire le token spécial <EOS> (End Of Sentence), ce qui dit à la machine : "La traduction est finie, tu peux t'arrêter !".

Si on n'utilise pas <EOS> le modèle ne va jamais s'arrêter ! Le modèle génère le mot <EOS> de lui-même parce qu'il a appris que toutes les phrases ont une fin.

---

# *FIN*

---